Rules vs LLMs Take-Home Assignment

Fire AI · Oct 21, 2026 · @Kaustubh Keshav

Overview

This assignment tests one skill: deciding when a deterministic rule is the right tool, when an LLM earns its cost, and when the answer is a combination of both. You will build two systems three ways each, measure them, and defend your final design.

There is no prize for using the most AI. A design that uses an LLM in only the few places it is genuinely better, and can prove it, will score highest.

Then, in a 60-minute interview, you will defend your choices. We will change requirements live and ask how your design copes.

AI tools are allowed and expected. Use LLMs, coding agents, Copilot, anything, to write your code. You must be able to explain and change every line you submit.

Item	Detail
Time window	2 calendar days from receiving the assignment
Language	Python 3.10+
Data	Public sources: Indian income-tax rules for Project A, NYC 311 service requests for Project B
LLM budget	Make use of free tier of Gemini
Submission	Private GitHub repo shared with the hiring team
Follow-up	60-minute defence interview within 5 days of submission

What we evaluate: the quality of your decisions about where to use rules and where to use LLMs, and the evidence behind them: correctness, cost, latency, auditability and how easily each part changes.

Project A — Income-tax engine: rules, LLM or both?

Your job is to build a service that tells a salaried person in India how much income tax they owe under the old and new regimes, which regime to choose, and exactly why. Users describe themselves in plain language, but the tax must be correct to the rupee. Then the

law changes, and we see which design copes.

Source material

- **Rules:** Income-tax provisions for salaried individuals for FY 2024-25 (AY 2025-26), from the Income Tax Department and the Finance Act 2024. Use official sources only.
- **Reference calculator:** the Income Tax Department's official tax calculator, for checking your results.
- **The change:** the Finance Act 2025 rules for FY 2025-26 (new-regime slabs, rebate and related changes), applied in Task 5.

Scope: salary income only. Cover the standard deduction, HRA exemption, Sections 80C, 80CCD(1B) and 80D (including senior-citizen parents), home-loan interest on a self-occupied house, the Section 87A rebate, surcharge with marginal relief, and the 4% health and education cess. Exclude capital gains, business income and other income heads.

Tasks

1. **Test suite first.** Build at least 200 test cases with expected tax under both regimes. Include edge cases around every threshold, and check at least 30 cases against the official calculator. Also write 50 plain-language user descriptions in varied styles, including Hinglish, incomplete and contradictory ones.
2. **Three implementations.** Build the same calculator three ways:
   a. **Hand-coded rules:** rules written by you as code or configuration.
   b. **LLM at query time:** an LLM reads the relevant rule text for each request and computes the answer.
   c. **LLM-compiled rules:** an LLM converts the rule text into executable rules once; you review them, and they then run without an LLM.
3. **Plain-language input.** Turn free-text user descriptions into structured inputs. When information is missing or ambiguous, the system must ask a clarifying question instead of guessing.
4. **Measure everything.** For each implementation, report rupee-exact accuracy, results repeated 10 times on the same input (is it deterministic?), latency, cost per 1,000 requests, and whether every rupee in the answer traces back to a specific clause.
5. **The law changes.** Apply the FY 2025-26 changes to all three implementations. Record how long each took, what broke, and which failures were silent (wrong answers with no error).
6. **Final design (max 2 pages).** Choose the architecture you would ship to 100,000 users, with each component labelled rule, LLM or hybrid, and justify every choice with your measurements. Explain how you would keep it correct through the next Budget.

Project B — Data-quality checks: rules, statistics or LLM review?

Your job is to build the data-quality layer that sits in front of Fire AI's dashboards: it must catch bad data before a CFO sees a wrong number, without drowning the team in false alarms or burning money on LLM calls. For each kind of data problem, you decide which kind of check belongs there, and prove it.

Dataset

NYC 311 Service Requests from NYC Open Data. Use requests created in calendar year 2023. The data mixes timestamps, categories, locations and free text (`Descriptor`, `Resolution Description`), and it already contains real quality problems for you to find.

Tasks

1. **Profile the raw data.** Find and document the quality problems that already exist in the 2023 data, with counts and examples.
2. **Build a corruption injector.** Write a tool that injects each of these 12 problem types into clean batches at a controlled rate, so detection can be measured:
   a. Renamed or dropped column
   b. Column type change (e.g. numbers arriving as strings)
   c. Date format switch partway through a batch
   d. Latitude and longitude swapped
   e. Exact duplicates
   f. Near-duplicates (same request, slightly altered timestamp or text)
   g. Null spike in one agency's records
   h. Closed date earlier than created date
   i. ZIP code inconsistent with borough
   j. `Complaint Type` inconsistent with `Descriptor`
   k. `Resolution Description` contradicting `Status`
   l. Personal data (phone numbers, emails, names) inside free-text fields
3. **Three detector families.** For every problem type, implement and measure (a) declarative rules or schema checks, (b) statistical or ML detectors, and (c) LLM review, either per row or on batch samples.
4. **Measure everything.** For each problem type and detector, report detection rate, false-alarm rate on clean batches, cost per million rows, throughput on a laptop, and how clearly each alert explains itself.

5. **Build the hybrid pipeline.** Combine the winners into one pipeline, for example cheap rules on every row, statistics per batch, and an LLM only on rows already flagged or sampled. Show what it catches and what it costs, end to end.
6. **Scale and design (max 2 pages).** Project the cost and latency of your pipeline at 50 million rows a day across 120 customers with different schemas. Which choices would change at that scale, and why?

Deliverables and submission rules

Submit one private repo in which a single command reproduces every number you report.

Deliverable	Format
Project A code	All three implementations, the plain-language parser and the test suite in `project_a/`
Project A results	`RESULTS_A.md`: the comparison table, the Task 5 change log (time taken, what broke, silent failures) and the final design
Project B code	The injector, all detectors and the hybrid pipeline in `project_b/`
Project B results	`RESULTS_B.md`: per-problem detection, false-alarm and cost table, the real issues found, and the scale design
Decision matrix	`DECISION_MATRIX.md`: one row per component or check, marked rule, statistical, LLM or hybrid, with the measurement that justifies it
AI usage note	`AI_USAGE.md`: which tools you used, for what, and one place where the AI was wrong and how you caught it

Rules

- Work alone. AI tools are fine; other people are not.
- Use only the sources named here.
- In the interview we will ask you to change your own code live, so make sure you understand all of it.

How you will be evaluated

The take-home and the defence interview are weighted equally. We reward choices that are backed by measurements, not the number of models or tools used.

Area	What we look for
A: Correctness	Rupee-exact results across the test suite, a thorough set of edge cases, and clarifying questions instead of guesses
A: Handling change	How quickly and safely each design absorbed the FY 2025-26 changes, and whether silent failures were caught
B: Detection quality	High detection with few false alarms across all 12 problem types, plus real issues found in the raw data
B: Cost discipline	LLM calls only where they clearly beat cheaper checks, with cost per million rows to prove it
Decision matrix	Every rule-vs-LLM choice tied to a measurement; trade-offs in auditability, determinism and maintenance stated honestly
Engineering quality	Runs with one command; clean structure and tests
Defence interview	Explaining each choice, adapting the design when requirements change, and changing your own code live